

A questo punto ho codificato un bit con 1000 bit
 se io devo codificare 1000 bit me ne servono un milione di bit.
 Mi piacerebbe avere un sistema che mi permetta di codificare 1 bit con 1 singolo bit

↳ PROBLEMA DEL LANCIO DI UNA MONETA SICURO

Se A, B onesti $\Rightarrow \text{coin}_A = \text{coin}_B$
 $\Pr[\text{coin}_A = 0] = \frac{1}{2}$
 se eseguono il programma che è stato stabilito per loro

$\left. \begin{matrix} \text{coin}_A \\ \text{coin}_B \end{matrix} \right\} \text{variabili con il risultato del lancio di una moneta}$

se solo uno dei due è disonesto, non possiamo dire più con certezza che $P = \frac{1}{2}$

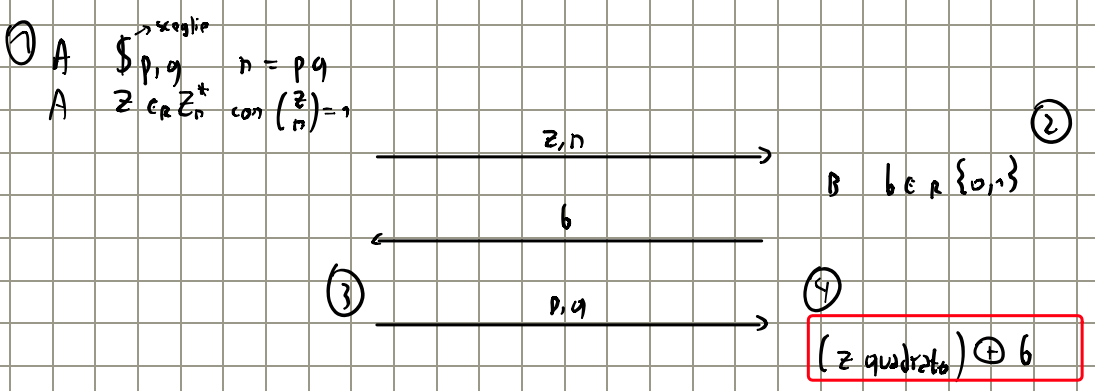
A ONESTA
 B DISONESTO $\Rightarrow \left| \Pr[\text{coin}_A = 0] - \frac{1}{2} \right| < \kappa^{-u(n)}$
 // vantaggio più piccolo di un qualsiasi polinomio

A DISONESTA
 B ONESTO $\Rightarrow \left| \Pr[\text{coin}_B = 0] - \frac{1}{2} \right| < \kappa^{-u(n)}$

↳ QUESTA COSA È SICURA?

Almeno dice, è una bella definizione

↳ PROTOCOLLO CHE SODDISFA LA PRECEDENTE DEFINIZIONE



// entrambi avranno lo stesso risultato

↳ COME SI PUÒ BARARE

Bob può mentire sulla randomicità del bit, quindi assegnando una certa probabilità a 0 e a 1. Ma questo non fa nessun effetto, lo xor è sempre casuale

Bob può cambiare b in funzione di z, ovvero calcolare quel bit b in maniera da alterare la probabilità che b sia uguale al quadrato di z. Ma questo equivale ad indovinare la quadraticità di z.

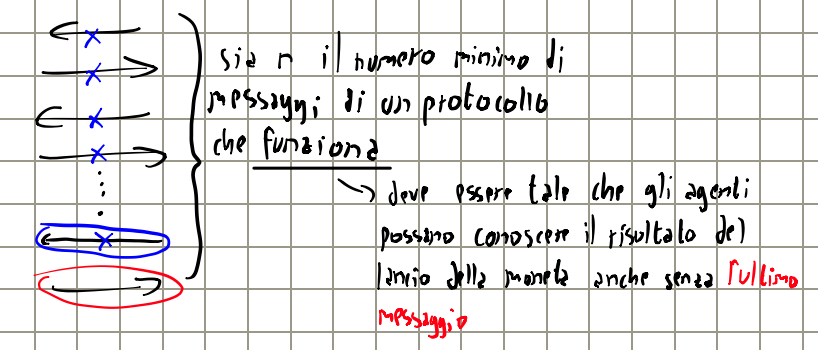
Alice potrebbe imbrogliare, nel caso scegliesse un z con simbolo di SACCA, 1 scelto in maniera non uniforme, Bob non è in grado di capire la modalità scelta di Alice. Ma Bob sceglie b comunque in maniera casuale con $P = \frac{1}{2}$, quindi Alice non riesce ad ottenere un vantaggio.

Alice può alterare in maniera pesante la probabilità, lo può fare rifiutandosi di inviare l'ultimo messaggio in base al coin B ricevuto da Bob, ovvero se coin A è sfavorevole per Alice allora quest'ultima non manda l'ultimo messaggio.

esempio: $\Pr[\text{coin}_B = 0] = \frac{1}{2} + \frac{1}{2} \cdot \left(\frac{1}{2}\right) = \frac{3}{4}$
 (favorevole ad A) (coin_A=0) (coin_A=1) $\rightarrow$ Bob lancia la propria moneta = 0

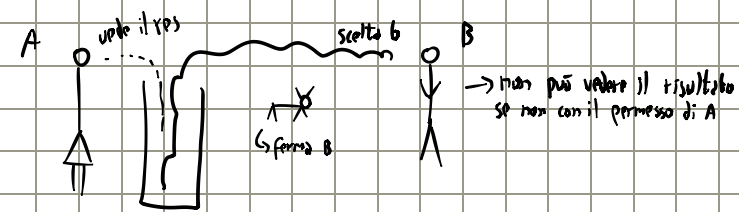
↳ QUINDI?

non esiste nessun protocollo che soddisfi la definizione



- l'ultimo messaggio non servirebbe a niente
- il penultimo messaggio è sempre un caso in cui B conosce e A non conosce
- A può sempre imbrogliare in quel caso
- Qualunque protocollo seguito, c'è sempre un messaggio che senza il quale una controparte conosce il risultato del lancio della moneta e l'altra parte non la conosce

↳ LANCIO DI MONETA NEL POZZO

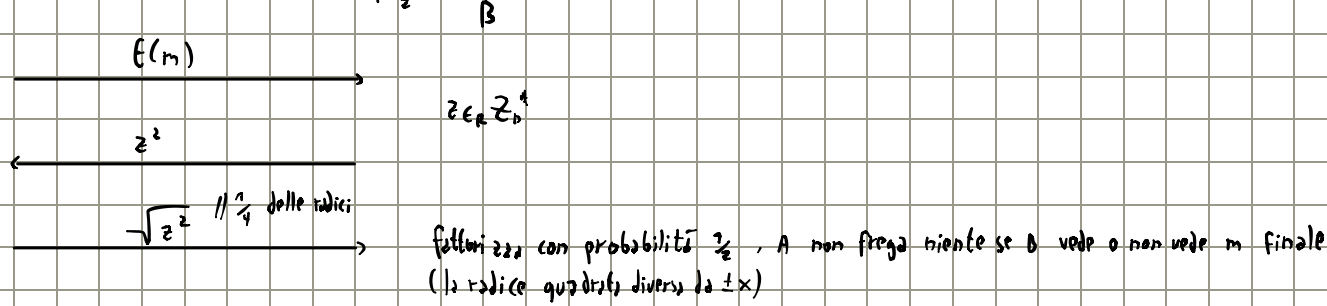


// non si può avere simmetria totale

↳ mini PROTOCOLLO PROPOSTO - OBLIVIOUS TRANSFER

RSA $n = p \cdot q$

il suo interesse non è l'outcome del risultato
ma che B riceve il risultato con $Pr \frac{1}{2}$



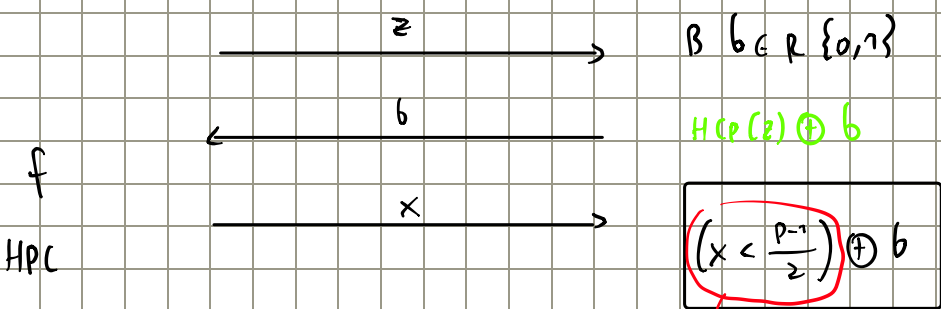
• il lancio del pozzo può far comodo

↳ HARD CORE PREDICATE DEL LOGARIMO DISCRETO

• Prendiamo un protocollo basato su logaritmo discreto

A $x \in \mathbb{Z}_{p-1}$

$z \leftarrow g^x$ // nasconde il valore di x (funzione one way) $z \leftarrow f(x)$



↳ dobbiamo dimostrare che è difficile, è un predicato binario sulla inversa della funzione che è difficile da calcolare. Ovvero scelto x a caso e poi si calcola g^x si è in grado di calcolare HCP (con una probabilità che si discosti da $\frac{1}{2}$ con una quantità più piccola di qualsiasi polinomio